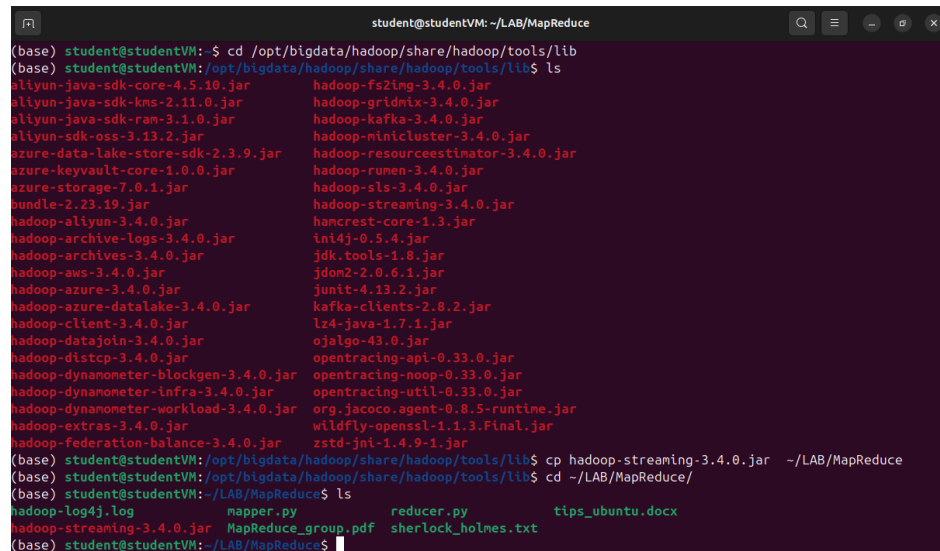


LAB2: MapReduce

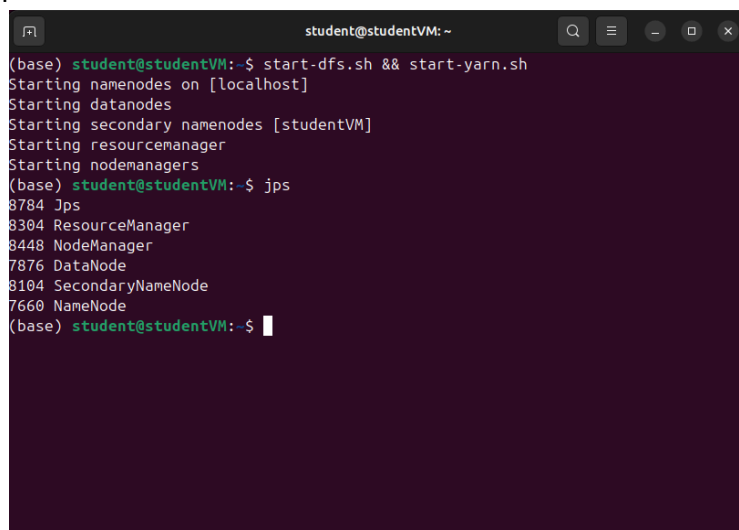
You've learnt basic linux commands from the last two labs. Today, we are going to introduce you to the core concept of data processing in a distributed file system called **MapReduce**. In summary, Map should be used to perform processing/filtering/sorting, Reduce is for aggregation and creating summary of mapped data. (see an example here: [MapReduce 101: What It Is & How to Get Started | Talend](#)) To understand how to use it, please follow along these steps before doing the lab section.

1. Copy `hadoop-streaming.jar` into lab MapReduce directory. The [hadoop-streaming.jar](#) is the utility file that allows you to create and run Map/Reduce jobs within a distributed file system. Our `hadoop-streaming.jar` is at `/opt/bigdata/hadoop/share/hadoop/tools/lib`.



```
student@studentVM: ~/LAB/MapReduce
(base) student@studentVM:~$ cd /opt/bigdata/hadoop/share/hadoop/tools/lib
(base) student@studentVM:/opt/bigdata/hadoop/share/hadoop/tools/lib$ ls
aliyun-java-sdk-core-4.5.10.jar      hadoop-fs2img-3.4.0.jar
aliyun-java-sdk-kms-2.11.0.jar      hadoop-gridmix-3.4.0.jar
aliyun-java-sdk-ran-3.1.0.jar       hadoop-kafka-3.4.0.jar
aliyun-sdk-oss-3.13.2.jar           hadoop-minicluster-3.4.0.jar
azure-data-lake-store-sdk-2.3.9.jar  hadoop-resourceestimator-3.4.0.jar
azure-keyvault-core-1.0.0.jar       hadoop-rumen-3.4.0.jar
azure-storage-7.0.1.jar            hadoop-sls-3.4.0.jar
bundle-2.23.19.jar                 hadoop-streaming-3.4.0.jar
hadoop-aliyun-3.4.0.jar             hamcrest-core-1.3.jar
hadoop-archive-logs-3.4.0.jar       ini4j-0.5.4.jar
hadoop-archives-3.4.0.jar           jdk.tools-1.8.jar
hadoop-aws-3.4.0.jar               jdom2-2.0.6.1.jar
hadoop-azure-3.4.0.jar             junit-4.13.2.jar
hadoop-azure-datalake-3.4.0.jar     kafka-clients-2.8.2.jar
hadoop-client-3.4.0.jar            lz4-java-1.7.1.jar
hadoop-datajoin-3.4.0.jar          ojalgo-43.0.jar
hadoop-distcp-3.4.0.jar            opentracing-api-0.33.0.jar
hadoop-dynamometer-blockgen-3.4.0.jar opentracing-noop-0.33.0.jar
hadoop-dynamometer-infra-3.4.0.jar  opentracing-util-0.33.0.jar
hadoop-dynamometer-workload-3.4.0.jar org.jacoco.agent-0.8.5-runtime.jar
hadoop-extras-3.4.0.jar           wildfly-openssl-1.1.3.Final.jar
hadoop-federation-balance-3.4.0.jar zstd-jni-1.4.9-1.jar
(base) student@studentVM:/opt/bigdata/hadoop/share/hadoop/tools/lib$ cp hadoop-streaming-3.4.0.jar ~/LAB/MapReduce/
(base) student@studentVM:/opt/bigdata/hadoop/share/hadoop/tools/lib$ cd ~/LAB/MapReduce/
(base) student@studentVM:~/LAB/MapReduce$ ls
hadoop-log4j.log      mapper.py      reducer.py      tips_ubuntu.docx
hadoop-streaming-3.4.0.jar  MapReduce_group.pdf  sherlock_holmes.txt
(base) student@studentVM:~/LAB/MapReduce$
```

2. Open your terminal in VMware and start a distributed file system and yarn process.
> `start-dfs.sh && start-yarn.sh`
Check everything has started already.
> `jps`



```
student@studentVM: ~
(base) student@studentVM:~$ start-dfs.sh && start-yarn.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [studentVM]
Starting resourcemanager
Starting nodemanagers
(base) student@studentVM:~$ jps
8784 Jps
8304 ResourceManager
8448 NodeManager
7876 DataNode
8104 SecondaryNameNode
7660 NameNode
(base) student@studentVM:~$
```

3. Assume that you want to count the word from sherlock_holmes.txt. In the concept of MapReduce: The mapper will scan through the file and output “I found that word 1 time” and the reducer will aggregate the sum of those “1 times” to total count.



```
mapper.py
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    line = line.strip()

    words = line.split()

    for word in words:
        print('%s\t%s'%(word, 1))

reducer.py
#!/usr/bin/env python3
import sys
from operator import itemgetter

current_word = None
current_count = 0

word = None

for line in sys.stdin:
    line = line.strip()

    word, count = line.split('\t', 1)

    try:
        count = int(count)
    except ValueError:
        continue

    if current_word == word:
        current_count += count
    else:
        if current_word:
            print('%s\t%d'%(current_word, current_count))
            current_count = count
            current_word = word

        if current_word == word:
            print('%s\t%d'%(current_word, current_count))
```

4. Put sherlock_holmes.txt into hadoop. Then, execute MapReduce command.

```
> hadoop fs -put ~/LAB/MapReduce/sherlock_holmes.txt
> hadoop jar hadoop-streaming-3.4.0.jar \
    -mapper ~/LAB/MapReduce/mapper.py \
    -reducer ~/LAB/MapReduce/reducer.py \
    -input sherlock_holmes.txt \
    -output result
```

```
(base) student@studentVM:~/LAB/MapReduce$ hadoop fs -ls
Found 1 items
-rw-r--r-- 1 student supergroup 2840502 2025-08-24 08:48 sherlock_holmes.txt
(base) student@studentVM:~/LAB/MapReduce$ hadoop jar hadoop-streaming-3.4.0.jar \
> -mapper ~/LAB/MapReduce/mapper.py \
> -reducer ~/LAB/MapReduce/reducer.py \
> -input sherlock_holmes.txt \
> -output result
packageJobJar: [/tmp/hadoop-unjar8756983053824029961/] [] /tmp/streamjob7787066847792725518.jar tmpDir=null
2025-08-24 09:02:32,706 INFO client.DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2025-08-24 09:02:32,891 INFO client.DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2025-08-24 09:02:33,088 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/
2025-08-24 09:02:33,308 INFO mapred.FileInputFormat: Total input files to process : 1
2025-08-24 09:02:33,771 INFO mapreduce.JobSubmitter: number of splits:2
```

Get the result into local storage for easier inspection.

```
2025-08-24 09:02:49,434 INFO streaming.StreamJob: Output directory: result
(base) student@studentVM:~/LAB/MapReduce$ hadoop fs -ls
Found 2 items
drwxr-xr-x 2 student supergroup 0 2025-08-24 09:02 result
-rw-r--r-- 1 student supergroup 2840502 2025-08-24 08:48 sherlock_holmes.txt
(base) student@studentVM:~/LAB/MapReduce$ hadoop fs -get result .
(base) student@studentVM:~/LAB/MapReduce$ ls
hadoop-log4j.log hadoop-streaming-3.4.0.jar mapper.py MapReduce_group.pdf reducer.py result sherlock_holmes.txt tips_ubuntu.docx
(base) student@studentVM:~/LAB/MapReduce$
```

NOTE: if you're not familiar with MapReduce with Hadoop, you can try MapRace in Pythonic way at first. It's easier to isolate how mapper and reducer are working. **But do not submit this pythonic MapReduce approach to us.**

For example; count totals of unique words as above in pythonic way:

```
> cat sherlock_holmes.txt | python3 mapper.py | sort -k 1 | python3 reducer.py
```

Lab assignment:

In MapReduce Folder, there is a “hadoop-log4j.log” java log file. This file contains 30MB of Hadoop and Yarn log messages including Timestamp, Log level, Source, and Details in each line.

```
2020-08-31 16:16:32,940 INFO org.apache.hadoop.hdfs.server.namenode.FSNamesystem: KeyProvider: null
2020-08-31 16:16:32,947 INFO org.apache.hadoop.hdfs.server.namenode.FSNamesystem: fsLock is fair: true
2020-08-31 16:16:32,955 INFO org.apache.hadoop.hdfs.server.namenode.FSNamesystem: Detailed lock hold time metrics enabled: false
2020-08-31 16:16:32,962 INFO org.apache.hadoop.hdfs.server.namenode.FSNamesystem: fsOwner = hdfs (auth:SIMPLE)
```

Your task is to edit mapper.py, reducer.py, MapReduce command within HDFS to acquire answers that fit following questions:

1. How many logs are there in each **log level** (INFO, WARN, ERROR) ?
2. How many logs are there in each type of **source** ?
3. **Which day** has highest number of logs occurred and how many ?
4. **Which source** has highest number of **ERROR level** logs occurred ?

Capture your edited codes, commands and results with explanation into a single pdf.